Decorator : is a design pattern that allows you to modify or extend the behaviour of a function without changing its source code.

Simply:
function that accepts another function as an argument, add some extra return new function

Ex:
```
@my_decorator
def say_Hello():
    print("Hello!")

def my_decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper
```

Manually reassigning
say_hello = my_deco(say_hello)
say_hello()

@ Coyntactic Sugary → right above target function

LLM Fundamentals

LLM → Large Language Model

ML model trained on large amounts of data to learn patterns in language, generate text, answer qn, perform other language related task

How LLM cook? Ex: Explain java inheritance

1. user promt
2. Tokenization (text is converted into token IDs)
3. Model processing (Neural network layers process input and context)
4. Token generation (Model predicts and select successive tokens)
5. Response (explanation)

Response not ready made answer from DB.

It generates response using its learned parameters

Context and strategy.

"Large" mean → scale of model, parameters
data used to train it

Parameters → learned numeric value inside a model

LLM understand like human?

No, learn patterns and relationship from training data and can produce responses

:→ LLM is powerful language model,
not an infallible source of truth

2. Tokens: How LLM processes Text
↓
Unit of text that particular model tokenizer recognize

Token may be: →Complete part of word, punctuation, number

Ex: "I love learning!" → token split (Tokenizer)
I / love / learning / ! ?

↓

An integer assigned to token (Token ID's): [15339, 1917, 1826]

↓

Numerical vectors enters model

Embedding → vectors representing token

Token affect:

Cost; Context limits, Performance

Input vs Output tokens

↓ ↓

Supplied to model, Generated by model.
including history & response
and Context ↳ long history consume many tokens if

3. Context window:

maximum amount of tokenized information a model can handle as context for req or generation

model's working context, not its permanent memory

Context window
- Instructions
- Conversation history
- Documents

} 8000 tokens

4. System Instructions

↳ Instruction supplied by the application or conversation setup to guide how the model should behave. Ex: Developer might want a chatbot to act as Java tutor who explains concepts

→ Establish role, style, beh

	Purpose	Example
System	high level behaviour	"beginner friendly Java tutor"
Developer	Provides application level instructions	"Return answer in specific format"
User	Provides Question	"Explain OOPS"
Assistant	Contains model resp	"OOPs contains four pillars..."
Tool	Contains result from external Tool	"search or DB Lookup"

User message: Question, command, document

Good prompt?:

1. Task: what should model do?
2. Context: what information does it need?
3. Constraint: what rules it must follow?
4. Output format: How answer should be presented?

Prompt vs System instruction

↓ establish overall assistant behaviour

↓ individual user request

6. Temperature: Controlling randomness in generation

↓

Decoding parameter supported by many language models

It changes model predicted token probabilities

Ex: model predicts next word in sentence

"The weather today is ..."

Possiblity of next word:

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Sunny, cloudy, pleasant, rainy

Model assign probabilities to possible next tokens based on context. Temperature influences how probabilities used during sampling.

Low vs High temperature

Concentrates probability on more likely options

Usually makes alternatives tokens more competitive during sampling

More predictable

More varied

7. Hallucinations → An LLM can give false info

↓

It occurs when the model produces information incorrect, fabricated or unsupported by evidence

Why hallucination happens?

1. Incomplete information:
2. Ambiguous questions: model makes incorrect assumption about user
3. Learned patterns: model generates plausible continuation not factually correct.
4. Missing context: Relevant documents or facts not provided
5. Poor retrieval: System retrieve irrelevant or incorrect info
6. Unverified tool result: accept external result without verify them

How to reduce hallucinations?

→ Provide relevant context

→ Use RAG where appropriate

→ Verify important claims

→ Allow uncertainty:

Inference → using a trained model

↓

Process using trained model to produce prediction or outputs for new inputs

Training:

Training data → model predictions → loss calculation

↓

Parameter update

Goal: Learn model parameter from data

Inference

New Input → trained model → generated output

Goal: Use trained model to produce result

Inference Latency:

time required to produce a response

For chatbot, it affected by:

→ model size and architecture

→ Input length and output length

→ Hardware and server load

→ Network latency, tools and api request

API Limits → request, quotas retry strategies

↳ control usage of service

Rate limit → Restrict usage over period

Ex: 60 req per minute

Token limit → Restrict token processing or generation

Ex: Maximum no. of tokens per request or minute

Quota: Restrict total available or usage

Ex: Monthly usage allowance

Application

↓

LLM API request

model, msg, parameter

authentication

↓

LLM Provider

Process req and generate output

↓

API response

Display respon

How all concepts fit together?

Building AI Agents:

User message
"Explains arrays"
↓
System Instructions
"Define tutor role and teaching style"
↓
Retrieval / Embeddings
"Find relevant teaching note"
↓
Tokenizer and LLM Inference
"Process context and generate tokens"
↓
Validation and app logic
"checks formating, applies rules and handles error"
↓
Final
"Explanation of arrays"

Tokens represent text

Context window constrain information supplied to model

System Instruction guide behaviour

Temperature influence generation

Embeddings help represent and retrieve info

Inference → generates output

Halluciation → Improve reliability